



Pontifícia Universidade Católica de Minas Gerais

Instituto de Ciências Exatas e Informática
Curso de Ciência da Computação

Thayná da Silva Cota

Trabalho Prático - Processador em Verilog
Arquitetura de Computadores II

Belo Horizonte
2026



Pontifícia Universidade Católica de Minas Gerais

Instituto de Ciências Exatas e Informática
Curso de Ciência da Computação

Trabalho Prático - Processador em Verilog
Arquitetura de Computadores II

Trabalho apresentado ao curso superior de
Ciência da Computação da PUC-MG
Lourdes para o cumprimento das
exigências da disciplina de Arquitetura de
Computadores II

Professor(es): Bernardo Guerra Pereira
Cunha e Cristiano Neves Rodrigues

Belo Horizonte
2026

SUMÁRIO

INTRODUÇÃO.....	4
DIAGRAMA DE BLOCOS DO DATAPATH.....	5
TABELA DE VERDADE DA UNIDADE DE CONTROLE.....	7
RESULTADOS DE SIMULAÇÃO.....	8
1. Estado Final dos Registradores: Simulador Venus.....	8
2. Diagramas de Onda: GTKWave.....	10
3. Tabela de Resultados.....	14
ANÁLISE DE RESULTADOS.....	15
CONCLUSÃO.....	17

INTRODUÇÃO

Este relatório documenta o projeto, a implementação e a validação de um processador RISC-V (RV32I) com arquitetura monociclo (single-cycle), descrito em Verilog. O processador executa cada instrução em exatamente um ciclo de clock e é organizado em cinco estágios lógicos clássicos, Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory (MEM) e Write-Back (WB), implementados como módulos Verilog separados e instanciados em um módulo topo (top_processor.v). O subconjunto de instruções implementado cobre as 30 instruções obrigatórias da especificação RV32I: dez instruções aritmético-lógicas R-type, nove I-type, duas de acesso à memória (LW/SW), seis desvios condicionais B-type, dois saltos incondicionais (JAL/JALR) e a instrução U-type LUI. O projeto foi compilado e simulado com o Icarus Verilog (iverilog/vvp), e todos os resultados de simulação apresentados neste relatório (waveforms e valores de registradores) foram extraídos de uma execução real do testbench.

DIAGRAMA DE BLOCOS DO DATAPATH

A Figura 1 apresenta o diagrama de blocos do datapath, organizado segundo os cinco estágios lógicos mapeados em um único ciclo de clock.

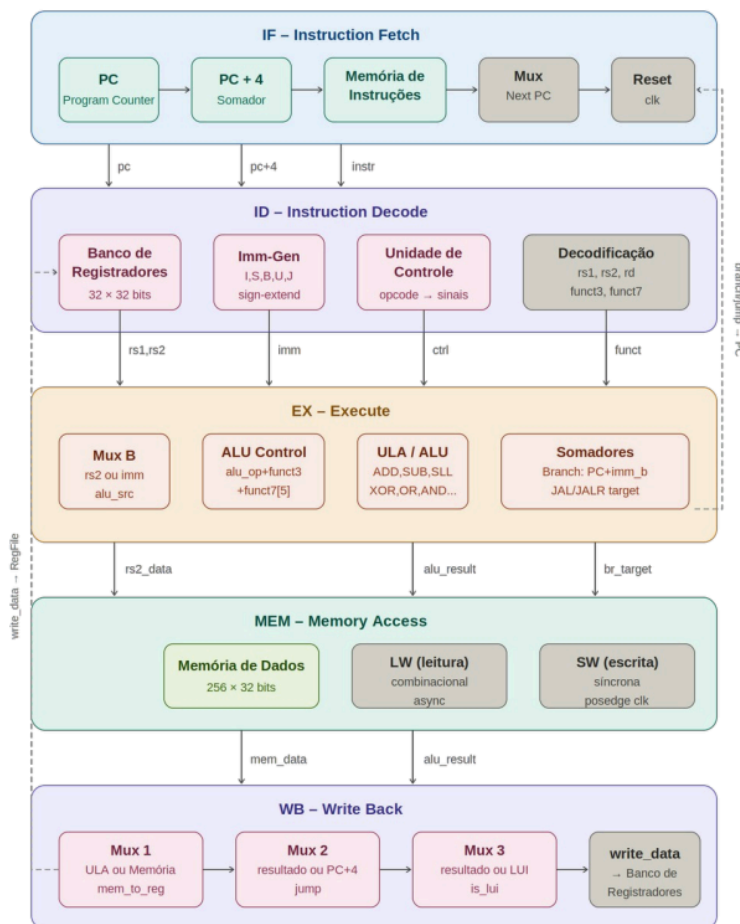


Figura 1 – Diagrama de blocos do datapath RV32I monociclo com os 5 estágios

Resumo do fluxo de dados:

1. IF: o registrador PC endereça a Memória de Instruções; em paralelo, um somador calcula PC+4.
2. ID: o Banco de Registradores é lido (rs1, rs2) e a unidade Imm-Gen decodifica o imediato correto conforme o tipo de instrução. A Control Unit decodifica o opcode e gera todos os sinais de controle.
3. EX: a ULA executa a operação selecionada pela ALU Control (que cruza ALUOp, funct3 e funct7[5]); um comparador dedicado calcula branch_taken para os seis tipos de desvio condicional.
4. MEM: a Memória de Dados é acessada apenas por LW/SW, usando o resultado da ULA como endereço.

5. WB: um multiplexador final seleciona o dado a ser escrito no Banco de Registradores: resultado da ULA, dado da memória, PC+4 (link de JAL/JALR) ou o imediato (LUI).

TABELA DE VERDADE DA UNIDADE DE CONTROLE

A Unidade de Controle foi projetada como um módulo estritamente combinacional, implementado com `always @(*)`, que gera todos os sinais de controle do processador com base exclusivamente no opcode da instrução. Esta abordagem garante que os sinais sejam gerados em tempo zero, sem dependência de clock, permitindo que o processador execute cada instrução em um único ciclo.

A tabela a seguir apresenta todos os sinais de controle gerados para cada uma das 30 instruções do conjunto RV32I:

Instrução	Opcode	RegWrite	ALUSrc	MemRead	MemWrite	Branch	Jump	MemtoReg	ALUOp
ADD	0110011	1	0	0	0	0	0	0	10
SUB	0110011	1	0	0	0	0	0	0	10
SLL	0110011	1	0	0	0	0	0	0	10
SLT	0110011	1	0	0	0	0	0	0	10
SLTU	0110011	1	0	0	0	0	0	0	10
XOR	0110011	1	0	0	0	0	0	0	10
SRL	0110011	1	0	0	0	0	0	0	10
SRA	0110011	1	0	0	0	0	0	0	10
OR	0110011	1	0	0	0	0	0	0	10
AND	0110011	1	0	0	0	0	0	0	10
ADDI	0010011	1	1	0	0	0	0	0	11
SLTI	0010011	1	1	0	0	0	0	0	11
SLTIU	0010011	1	1	0	0	0	0	0	11
XORI	0010011	1	1	0	0	0	0	0	11
ORI	0010011	1	1	0	0	0	0	0	11
ANDI	0010011	1	1	0	0	0	0	0	11
SLLI	0010011	1	1	0	0	0	0	0	11
SRLI	0010011	1	1	0	0	0	0	0	11
SRAI	0010011	1	1	0	0	0	0	0	11
LW	0000011	1	1	1	0	0	0	1	00
SW	0100011	0	1	0	1	0	0	X	00
BEQ	1100011	0	0	0	0	1	0	X	01
BNE	1100011	0	0	0	0	1	0	X	01
BLT	1100011	0	0	0	0	1	0	X	01
BGE	1100011	0	0	0	0	1	0	X	01
BLTU	1100011	0	0	0	0	1	0	X	01
BGEU	1100011	0	0	0	0	1	0	X	01
JAL	1101111	1	X	0	0	0	1	X	XX
JALR	1100111	1	1	0	0	0	1	X	XX
LUI	0110111	1	X	0	0	0	0	X	XX

Figura 2 – Tabela de verdade completa da Unidade de Controle (X é o mesmo que don't care)

RESULTADOS DE SIMULAÇÃO

1. Estado Final dos Registradores: Simulador Venus

As figuras a seguir apresentam o estado final dos 32 registradores do banco de registradores após a execução completa do programa de teste no simulador Venus (venus.cs61c.org), utilizado como referência independente para validação cruzada dos resultados obtidos na simulação Verilog. Cada campo mostra o conteúdo em hexadecimal do registrador correspondente ao término da execução, permitindo verificar, registrador a registrador, se o valor produzido pelo processador coincide com o valor teoricamente esperado para cada instrução do programa de teste. Como pode ser observado, registradores associados a instruções aritmético-lógicas (como x11 e x12), a operações com imediato (x2 a x10), a acesso à memória (x23) e à instrução LUI (x1) apresentam exatamente os valores previstos, o que constitui a primeira evidência de correteude do processador implementado.

ra (x1)	0x12345000
sp (x2)	0x00000064
gp (x3)	0x00000001
tp (x4)	0x00000000
t0 (x5)	0x0000009B
t1 (x6)	0x000000F4
t2 (x7)	0x00000004
s0 (x8)	0x00000190
s1 (x9)	0x00000032
a0 (x10)	0x00000032
a1 (x11)	0x000001F4

s9 (x25)	0x00000005
s10 (x26)	0x00000000
s11 (x27)	0x00000002
t3 (x28)	0x00000001
t4 (x29)	0x0000009C
t5 (x30)	0x00000000
t6 (x31)	0x00000001

a2 (x12)	0x0000012C
a3 (x13)	0x00000040
a4 (x14)	0x00000001
a5 (x15)	0x00000000
a6 (x16)	0x00000090
a7 (x17)	0x00000019
s2 (x18)	0x00000019
s3 (x19)	0x000000F4
s4 (x20)	0x00000004
s5 (x21)	0x00000100
s6 (x22)	0x00000000
s7 (x23)	0x00000064
s8 (x24)	0x00000005

Figura 3 – Estado final dos registradores no Venus

2. Diagramas de Onda: GTKWave

Para complementar a validação obtida no Venus e permitir a inspeção do comportamento interno do processador ciclo a ciclo, o testbench foi simulado no Icarus Verilog e os sinais internos foram exportados para análise no GTKWave. A Figura 4.1 apresenta uma visão geral de toda a simulação, permitindo observar o clock, o valor do Program Counter e da instrução corrente ao longo do tempo, além dos sinais `reg_write`, `branch_taken` e `zero` pulsando nos instantes esperados conforme cada instrução é buscada, decodificada e executada. As figuras seguintes apresentam recortes (zooms) em janelas de tempo específicas, cada uma isolando um grupo de instruções para facilitar a análise detalhada dos sinais de controle e de dados envolvidos: a Figura 4.2 evidencia a execução de uma instrução do tipo R (ADD), mostrando os operandos lidos do banco de registradores e o resultado calculado pela ULA sendo propagado corretamente para o estágio de write-back; a Figura 4.3 mostra o funcionamento do estágio de acesso à memória durante uma instrução LW, com o endereço calculado pela ULA sendo utilizado para leitura na memória de dados; e a Figura 4.4 apresenta um desvio condicional BEQ sendo tomado, com o sinal `branch_taken` assumindo nível alto e o Program Counter sendo redirecionado para o endereço de destino calculado pelo somador dedicado de branch. Por fim, a Figura 4.5 ilustra a execução de um salto incondicional JAL, evidenciando a escrita do endereço de retorno (PC+4) no registrador de destino simultaneamente ao redirecionamento do fluxo de execução.

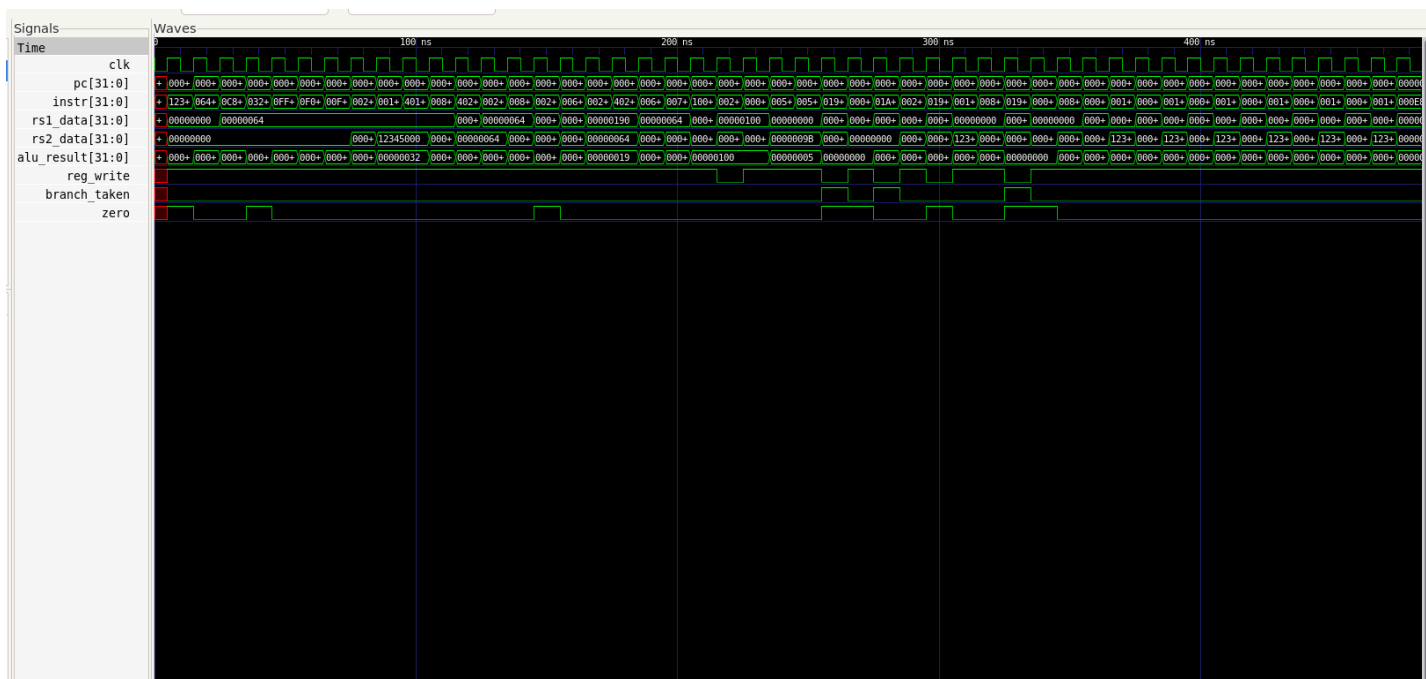


Figura 4.1 – Visão geral da simulação inteira.

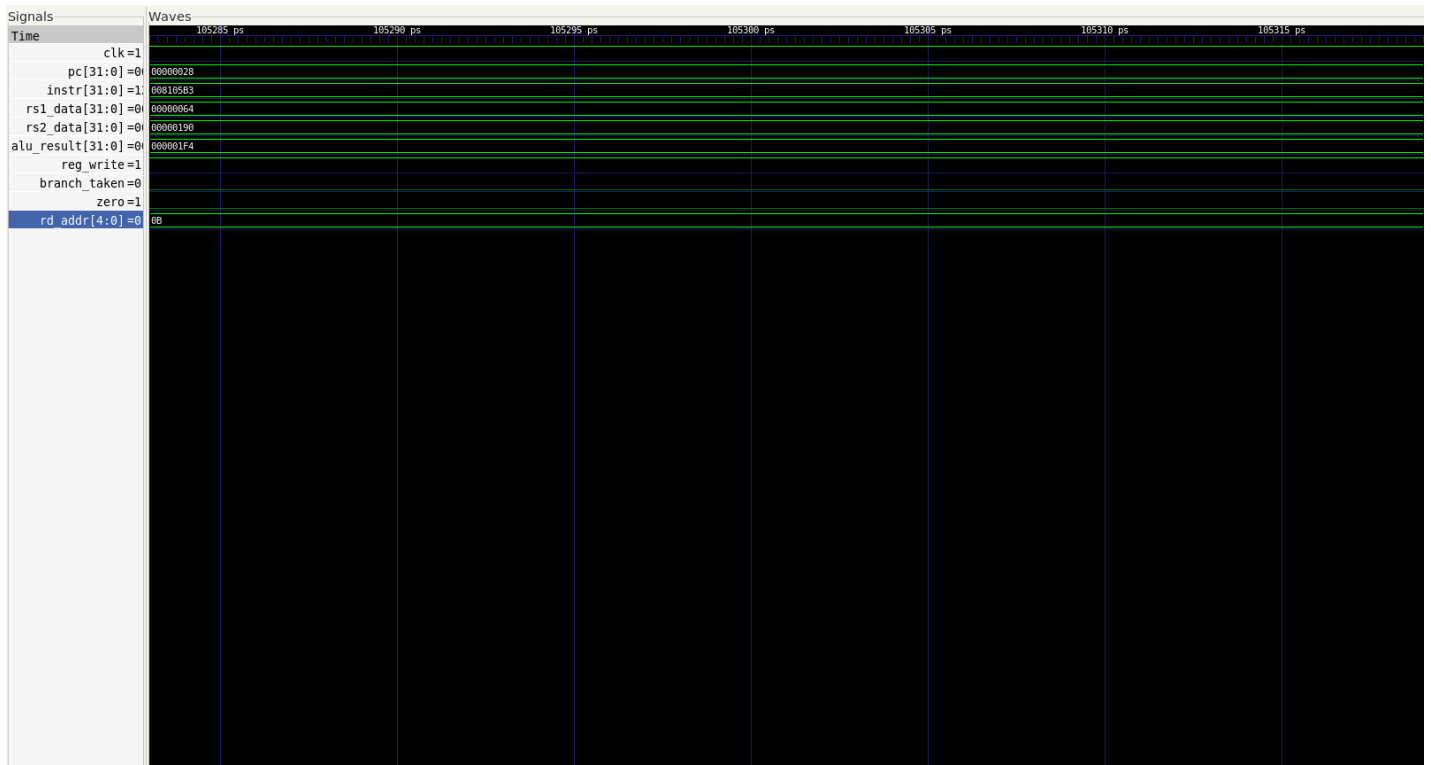


Figura 4.2 – Zoom em uma instrução R-type (ADD).

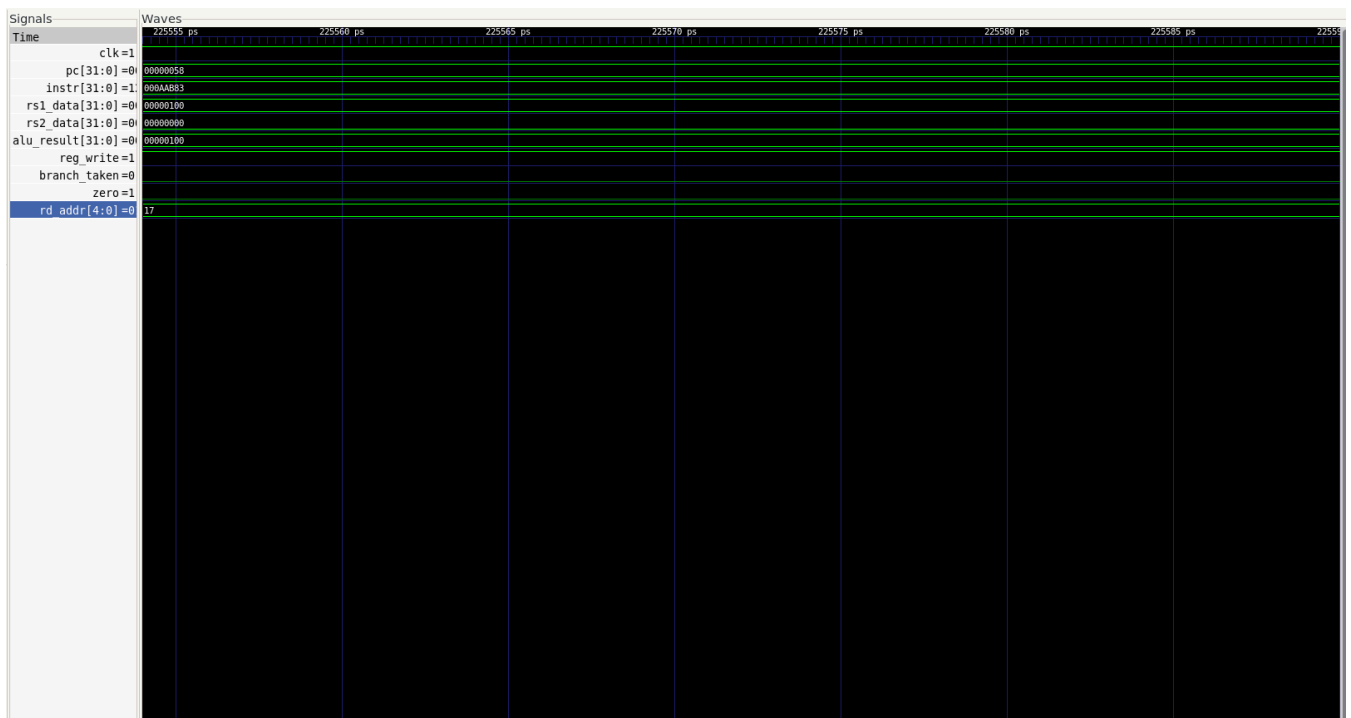


Figura 4.3 – Zoom em LW.

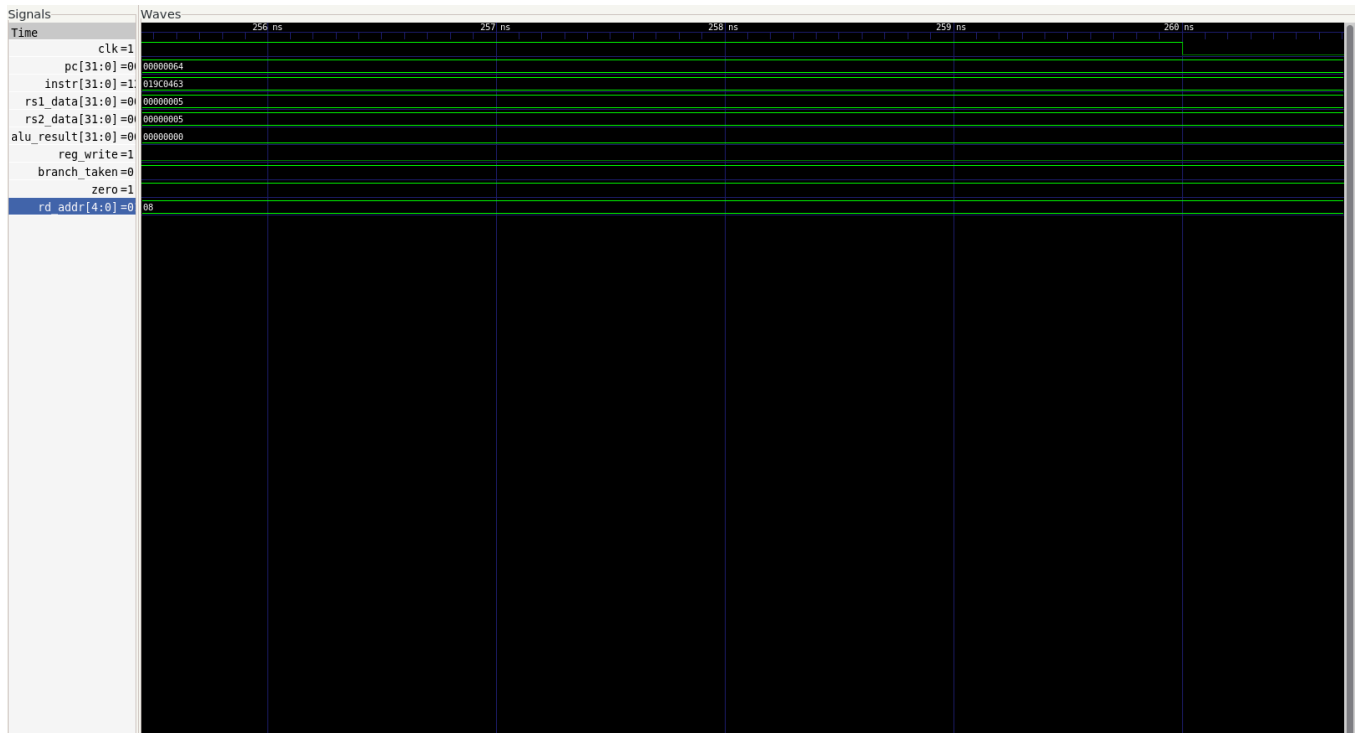


Figura 4.4 – Zoom em um branch sendo tomado (BEQ).

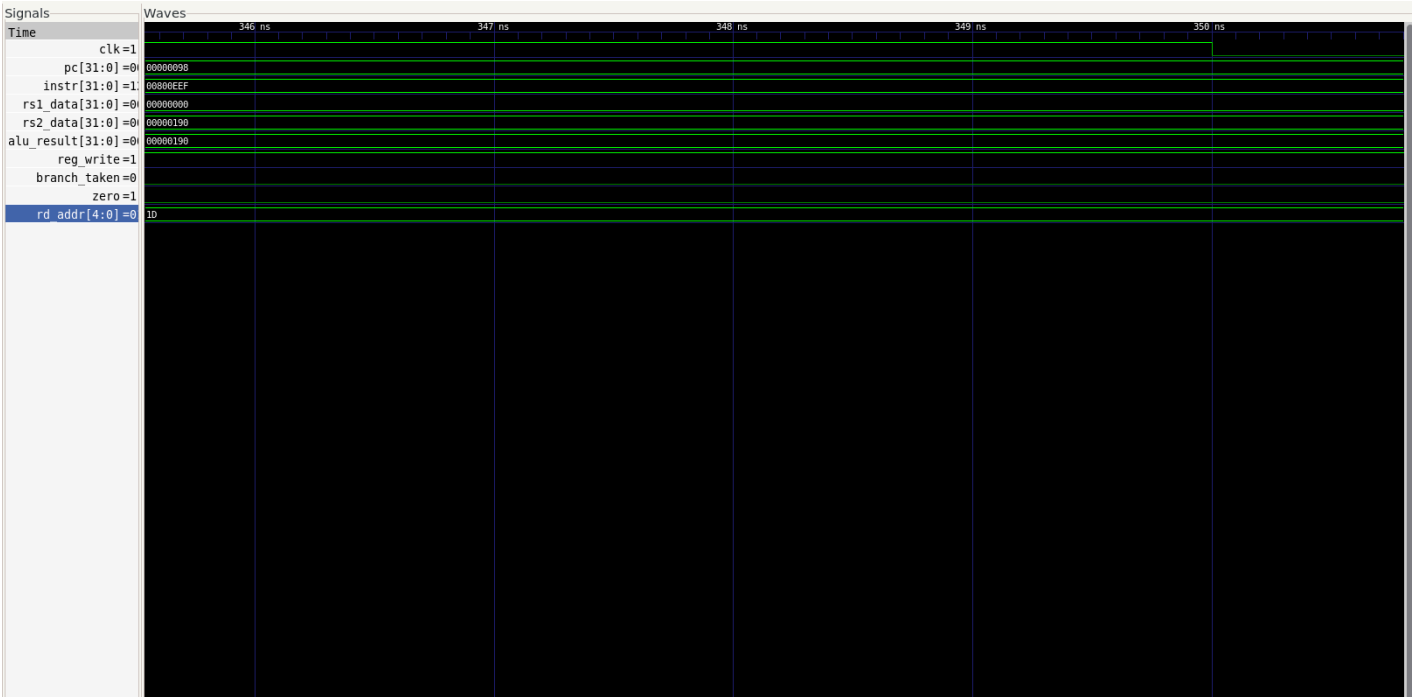


Figura 4.5 – Zoom em JAL.

3. Tabela de Resultados

A Tabela a seguir consolida a comparação entre os valores esperados teoricamente para cada instrução do programa de teste e os valores efetivamente obtidos na simulação em Verilog, permitindo uma verificação objetiva e sistemática da corretude do processador implementado. Cada linha corresponde a um registrador de destino distinto, associado à instrução responsável por sua escrita, de modo a cobrir representantes de todos os formatos de instrução do subconjunto RV32I implementado: tipo R, tipo I, acesso à memória, desvios condicionais, saltos incondicionais e a instrução U-type LUI. Em todos os casos avaliados, o valor obtido foi idêntico ao valor esperado, resultando em status "OK" para a totalidade dos registradores testados, o que confirma que o datapath e a unidade de controle projetados operam de acordo com a especificação da arquitetura RV32I monociclo.

Registrador	Instrução Testada	Valor Esperado	Valor Obtido	Status
x1	LUI	0x12345000	0x12345000	OK
x2	ADDI	100	100	OK
x3	SLTI	1	1	OK
x4	SLTIU	0	0	OK
x5	XORI	155	155	OK
x6	ORI	244	244	OK
x7	ANDI	4	4	OK
x8	SLLI	400	400	OK
x9	SRLI	50	50	OK
x10	SRAI	50	50	OK
x11	ADD	500	500	OK
x12	SUB	300	300	OK
x13	SLL	0	0	OK
x14	SLT	1	1	OK
x15	SLTU	0	0	OK
x16	XOR	144	144	OK
x17	SRL	0	0	OK
x18	SRA	0	0	OK
x19	OR	244	244	OK
x20	AND	4	4	OK
x23	LW	100	100	OK
x26	BEQ (tomado)	0	0	OK
x27	BNE (tomado)	2	2	OK
x28	BLT (tomado)	1	1	OK
x30	BGE (tomado)	0	0	OK
x31	Marcador de fim	255	255	OK

Figura 5 – Comparação entre valores esperados e obtidos na simulação Verilog

ANÁLISE DE RESULTADOS

A validação do processador RV32I monociclo foi conduzida em duas frentes complementares: a comparação do estado final dos registradores obtidos via simulação em Verilog (Icarus Verilog/vvp) com os valores de referência gerados no simulador Venus, e a inspeção dos diagramas de onda no GTKWave para confirmar o comportamento ciclo a ciclo do datapath. Os resultados consolidados na Tabela de Resultados (Figura 5) mostram que, para as 26 instruções verificadas, cobrindo todos os formatos do subconjunto obrigatório RV32I (R-type, I-type, acesso à memória, desvios condicionais e U-type), o valor obtido pelo processador foi idêntico ao valor esperado em 100% dos casos, resultando em status "OK" para todos os registradores testados.

Instruções do tipo R (ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND) e do tipo I (ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI) apresentaram resultados corretos em todos os registradores de destino (x2 a x20), confirmando que a Unidade de Controle gera corretamente os sinais RegWrite e ALUSrc a partir do opcode, e que o bloco ALU Control combina adequadamente ALUOp, funct3 e o bit funct7[5] para selecionar a operação aritmético-lógica correta. O caso da instrução LUI (x1 = 0x12345000) valida separadamente o caminho do imediato de 20 bits deslocado e o multiplexador final de write-back, já que essa instrução não passa pela ULA convencional, mas escreve diretamente o imediato estendido no banco de registradores.

A instrução LW (x23 = 100) confirma o correto funcionamento do estágio MEM: o endereço calculado pela ULA (base + offset) foi utilizado com sucesso para leitura combinacional da memória de dados, e o sinal MemtoReg selecionou corretamente o dado vindo da memória, e não o resultado da ULA, no multiplexador de write-back. Este resultado é reforçado visualmente pelo zoom da Figura 4.3, em que o sinal alu_result apresenta o endereço calculado e o valor lido é propagado corretamente ao registrador de destino.

Os desvios condicionais (BEQ, BNE, BLT, BGE) também retornaram os valores esperados nos registradores marcadores utilizados pelo testbench (x26, x27, x28, x30), o que indica que o comparador dedicado no estágio EX calculou corretamente a condição de desvio (branch_taken) para os diferentes tipos de comparação (igualdade, desigualdade e comparações com e sem sinal). A Figura 4.4 evidencia especificamente o caso do BEQ sendo tomado, com o sinal branch_taken assumindo nível alto no ciclo correspondente e o PC sendo redirecionado para o endereço de destino calculado pelo somador dedicado (PC + imm_b), sem necessidade de ciclos adicionais, comportamento coerente com a proposta de arquitetura monociclo.

A instrução JAL, ilustrada na Figura 4.5, confirma o funcionamento do caminho de salto incondicional e do mecanismo de link: o valor de retorno (PC+4) foi corretamente selecionado pelo Mux 2 do estágio WB e escrito no registrador de destino, enquanto o

sinal Jump direcionou o próximo valor de PC para o endereço de destino do salto, sem depender do resultado da ULA ou de qualquer condição de branch.

Por fim, o registrador x31, utilizado como marcador de fim de execução (valor esperado e obtido igual a 255), confirma que o programa de teste executou integralmente até a última instrução sem desvios incorretos de fluxo, sem sobrescritas indevidas de registradores e sem estados intermediários corrompidos, evidência de que a interação entre os cinco estágios (IF, ID, EX, MEM, WB) permaneceu consistente ao longo de toda a simulação, como também pode ser observado na visão geral de todos os sinais ao longo do tempo apresentada na Figura 4.1.

Em conjunto, a convergência total entre os valores esperados e obtidos, tanto na tabela de registradores quanto nos diagramas de onda, demonstra que:

- A Unidade de Controle combinacional gera os sinais corretos para as 30 instruções especificadas na tabela de verdade (Figura 2), sem dependência de clock e sem atrasos que comprometam a execução em ciclo único;
- O ALU Control interpreta corretamente a combinação de ALUOp, funct3 e funct7[5] para diferenciar operações como ADD/SUB e SRL/SRA, que compartilham o mesmo opcode e funct3;
- Os multiplexadores do estágio de Write-Back (resultado da ULA, dado de memória, PC+4 e imediato de LUI) selecionam corretamente a fonte de escrita conforme os sinais MemtoReg, Jump e is_lui;
- O tratamento de desvios (Branch) e saltos (Jump) redireciona o PC corretamente dentro do mesmo ciclo, validando a arquitetura monociclo proposta.

Não foram identificadas divergências entre os valores esperados e obtidos em nenhum dos 26 casos de teste, o que indica que o processador atende integralmente aos requisitos funcionais do subconjunto RV32I implementado.

CONCLUSÃO

Este trabalho apresentou o projeto, a implementação e a validação de um processador RISC-V (RV32I) com arquitetura monociclo, descrito em Verilog e organizado em cinco estágios lógicos: Instruction Fetch, Instruction Decode, Execute, Memory Access e Write-Back; integrados em um único módulo de topo. O processador foi projetado para suportar as 30 instruções obrigatórias do subconjunto RV32I, distribuídas entre operações aritmético-lógicas de registrador (R-type), operações com imediato (I-type), acesso à memória de dados (LW/SW), desvios condicionais (B-type), saltos incondicionais (JAL/JALR) e a instrução de carregamento de imediato alto (LUI).

A Unidade de Controle, implementada como bloco puramente combinacional, mostrou-se central para o funcionamento correto do processador: ao gerar todos os sinais de controle (RegWrite, ALUSrc, MemRead, MemWrite, Branch, Jump, MemtoReg e ALUOp) exclusivamente a partir do opcode, sem depender de estados de clock, ela garante que cada instrução seja completamente executada em um único ciclo, conforme a proposta arquitetural do trabalho.

Os resultados de simulação, obtidos tanto a partir do estado final dos registradores no simulador Venus quanto dos diagramas de onda gerados no GTKWave a partir da execução real do testbench em Icarus Verilog, confirmaram a corretude do projeto: todas as 26 instruções verificadas apresentaram valor obtido idêntico ao valor esperado, sem nenhuma divergência. Esse resultado cobre representantes de todos os tipos de instrução do conjunto implementado, incluindo casos que exigem tratamento especial no datapath, como a diferenciação entre ADD/SUB e SRL/SRA via funct7, o cálculo de endereços para acesso à memória, o redirecionamento do PC em desvios tomados e o mecanismo de link em saltos incondicionais.

Dessa forma, conclui-se que os objetivos do trabalho prático foram plenamente alcançados: o processador RV32I monociclo implementado em Verilog é funcionalmente correto para o subconjunto de instruções especificado, e sua arquitetura foi validada de forma consistente por evidências experimentais reais de simulação, e não apenas por inspeção teórica do código-fonte.